

CS 301: Advanced Operating Systems & Distributed Architecture Lecture 7:
Concurrency, Virtual Memory, and Distributed Consensus

1. **PROCESS SYNCHRONIZATION AND RACE CONDITIONS** In multi-threaded execution environments, concurrent access to shared resources without proper synchronization leads to race conditions. A race condition occurs when the system's substantive behavior depends on the uncontrollable order or timing of thread scheduling.

The critical section problem requires four conditions for a correct solution: - Mutual Exclusion: If thread T1 is executing in its critical section, no other threads may be executing in their critical sections. - Progress: If no thread is in its critical section and some threads wish to enter, only those threads not in remainder sections can participate in deciding who enters next. - Bounded Waiting: A bound must exist on the number of times other threads are allowed to enter their critical sections after a thread has requested entry. - Architectural Independence: No assumptions can be made regarding hardware processing speeds.

Classical synchronization primitives include mutex locks, counting semaphores, and condition variables. Mutexes provide strict binary ownership, whereas counting semaphores maintain an integer counter decremented by `wait()` / `P()` and incremented by `signal()` / `V()`. When the counter reaches zero, subsequent acquiring threads are transitioned to a blocked wait queue, avoiding CPU spin-wait degradation.

2. **VIRTUAL MEMORY ARCHITECTURE AND PAGE FAULT HANDLING** Virtual memory decouples the programmer's logical address space from physical random-access memory (RAM). The Memory Management Unit (MMU) performs hardware translation using Multi-Level Page Tables.

On an x86-64 architecture, a 48-bit canonical virtual address is translated across 4 levels: - PML4 (Page Map Level 4) - PDPT (Page Directory Pointer Table) - PD (Page Directory) - PT (Page Table) - Offset (12 bits addressing a 4KB physical page frame)

Translation Lookaside Buffers (TLB) cache recent virtual-to-physical translations to eliminate the high latency penalty of walking four levels of DRAM page tables per memory access. A TLB hit resolves in ~1 nanosecond, whereas a TLB miss incurs a 50-100 nanosecond memory bus traversal penalty.

When an accessed virtual page is not marked present in physical RAM (valid bit = 0), the MMU raises an architectural trap: a Page Fault (Exception 14). The OS page fault handler executes the following deterministic sequence: 1. Save processor state and determine the faulting memory address from the CR2 register. 2. Verify that the memory reference was valid within the process's Virtual Memory Area (VMA). 3. Select a physical victim frame using a page replacement algorithm (such as Least Recently Used - LRU, or Clock / Second-Chance). 4. If the victim page is dirty, write back modified contents to the

swap partition on secondary disk storage. 5. Issue an asynchronous DMA read request to page in the requested disk block to the newly allocated physical frame. 6. Update the Page Table Entry (PTE) with the physical frame address and set the valid bit to 1. 7. Restore register context and restart the faulting CPU instruction.

Thrashing describes the pathological condition where CPU utilization collapses because processes spend virtually all execution time swapping pages into and out of secondary storage rather than executing application logic. It occurs when the sum of working sets across all active processes exceeds physical memory capacity.

3. DISTRIBUTED SYSTEMS CONSENSUS: RAFT ALGORITHM Distributed consensus ensures that a collection of independent nodes agree on a shared sequence of state machine transitions, even under partial network partitions and server crashes (omission failures).

The Raft consensus algorithm decomposes state machine replication into three orthogonal subproblems: - Leader Election: When an existing leader fails or heartbeats timeout, a follower increments its currentTerm and transitions to the Candidate state, requesting votes across the cluster. If it gains a majority ($N/2 + 1$), it becomes the leader for that term. - Log Replication: The elected leader accepts client requests, appends log entries locally, and issues AppendEntries RPCs to all followers. An entry is considered committed once safely replicated across a cluster majority. - Safety Invariant: Raft guarantees that if a leader commits a log entry at a specific index and term, no other server will ever commit a different entry for that log position. Candidates with outdated logs are rejected during leader election voting.

By maintaining state machine invariants across majorities (quorums), Raft achieves consensus without single points of failure.